

part_b_deep_learning

January 21, 2026

PART B: DEEP LEARNING VIDEO CLASSIFICATION (REAL DATA)

This script implements: 1. 2D CNN (ResNet-18) + Temporal Pooling 2. 3D CNN (R(2+1)D-18)

Dataset: - UCF-style directory - Predefined train/test splits

Evaluation: - Accuracy - Precision (macro) - Recall (macro) - F1-score (macro) - Confusion Matrix
- Training time - Inference time per video

Author: 2024ab05275

```
[11]: pip install -r requirements.txt
```

```
Requirement already satisfied: numpy>=1.23 in /opt/conda/lib/python3.11/site-  
packages (from -r requirements.txt (line 23)) (1.24.4)  
Requirement already satisfied: scipy>=1.10 in /opt/conda/lib/python3.11/site-  
packages (from -r requirements.txt (line 31)) (1.11.3)  
Requirement already satisfied: torch>=2.0 in /opt/conda/lib/python3.11/site-  
packages (from -r requirements.txt (line 42)) (2.3.0+cu121)  
Requirement already satisfied: torchvision>=0.15 in  
/opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 49))  
(0.18.0+cu121)  
Requirement already satisfied: openpyxl>=3.1.0 in  
/opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 54))  
(3.1.5)  
Requirement already satisfied: opencv-python-headless in  
/opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 69))  
(4.11.0.86)  
Requirement already satisfied: scikit-learn>=1.3 in  
/opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 77))  
(1.8.0)  
Requirement already satisfied: scikit-image in /opt/conda/lib/python3.11/site-  
packages (from -r requirements.txt (line 84)) (0.22.0)  
Requirement already satisfied: matplotlib>=3.7 in  
/opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 96))  
(3.8.4)  
Requirement already satisfied: seaborn>=0.13 in /opt/conda/lib/python3.11/site-  
packages (from -r requirements.txt (line 103)) (0.13.2)  
Requirement already satisfied: umap-learn in /opt/conda/lib/python3.11/site-  
packages (from -r requirements.txt (line 110)) (0.5.11)
```

Requirement already satisfied: tqdm>=4.66 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 122)) (4.66.4)

Requirement already satisfied: filelock in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.13.1)

Requirement already satisfied: typing-extensions>=4.8.0 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (4.12.2)

Requirement already satisfied: sympy in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (1.12)

Requirement already satisfied: networkx in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.3)

Requirement already satisfied: jinja2 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.1.4)

Requirement already satisfied: fsspec in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (2024.2.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cudnn-cu12==8.9.2.26 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (8.9.2.26)

Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.3.1)

Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (11.0.2.54)

Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (10.3.2.106)

Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (11.4.5.107)

Requirement already satisfied: nvidia-cuspars-cu12==12.1.0.106 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.0.106)

Requirement already satisfied: nvidia-nccl-cu12==2.20.5 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (2.20.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: triton==2.3.0 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (2.3.0)

Requirement already satisfied: nvidia-nvjitlink-cu12 in /opt/conda/lib/python3.11/site-packages (from nvidia-cusolver-cu12==11.4.5.107->torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /opt/conda/lib/python3.11/site-packages (from torchvision>=0.15->-r requirements.txt (line 49)) (10.4.0)

Requirement already satisfied: et-xmlfile in /opt/conda/lib/python3.11/site-packages (from openpyxl>=3.1.0->-r requirements.txt (line 54)) (2.0.0)

Requirement already satisfied: joblib>=1.3.0 in /opt/conda/lib/python3.11/site-packages (from scikit-learn>=1.3->-r requirements.txt (line 77)) (1.4.2)

Requirement already satisfied: threadpoolctl>=3.2.0 in /opt/conda/lib/python3.11/site-packages (from scikit-learn>=1.3->-r requirements.txt (line 77)) (3.5.0)

Requirement already satisfied: imageio>=2.27 in /opt/conda/lib/python3.11/site-packages (from scikit-image->-r requirements.txt (line 84)) (2.34.2)

Requirement already satisfied: tifffile>=2022.8.12 in /opt/conda/lib/python3.11/site-packages (from scikit-image->-r requirements.txt (line 84)) (2024.7.2)

Requirement already satisfied: packaging>=21 in /opt/conda/lib/python3.11/site-packages (from scikit-image->-r requirements.txt (line 84)) (24.1)

Requirement already satisfied: lazy_loader>=0.3 in /opt/conda/lib/python3.11/site-packages (from scikit-image->-r requirements.txt (line 84)) (0.4)

Requirement already satisfied: contourpy>=1.0.1 in /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r requirements.txt (line 96)) (1.2.1)

Requirement already satisfied: cycler>=0.10 in /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r requirements.txt (line 96)) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r requirements.txt (line 96)) (4.53.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r requirements.txt (line 96)) (1.4.5)

Requirement already satisfied: pyparsing>=2.3.1 in /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r requirements.txt (line 96)) (3.1.2)

Requirement already satisfied: python-dateutil>=2.7 in /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r requirements.txt (line 96)) (2.9.0)

Requirement already satisfied: pandas>=1.2 in /opt/conda/lib/python3.11/site-packages (from seaborn>=0.13->-r requirements.txt (line 103)) (2.1.4)

Requirement already satisfied: numba>=0.51.2 in /opt/conda/lib/python3.11/site-packages (from umap-learn->-r requirements.txt (line 110)) (0.63.1)

Requirement already satisfied: pynndescent>=0.5 in /opt/conda/lib/python3.11/site-packages (from umap-learn->-r requirements.txt

(line 110)) (0.6.0)
Requirement already satisfied: llvmlite<0.47,>=0.46.0dev0 in
/opt/conda/lib/python3.11/site-packages (from numba>=0.51.2->umap-learn->-r
requirements.txt (line 110)) (0.46.0)
Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.11/site-
packages (from pandas>=1.2->seaborn>=0.13->-r requirements.txt (line 103))
(2024.1)
Requirement already satisfied: tzdata>=2022.1 in /opt/conda/lib/python3.11/site-
packages (from pandas>=1.2->seaborn>=0.13->-r requirements.txt (line 103))
(2024.1)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.11/site-
packages (from python-dateutil>=2.7->matplotlib>=3.7->-r requirements.txt (line
96)) (1.16.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/opt/conda/lib/python3.11/site-packages (from jinja2->torch>=2.0->-r
requirements.txt (line 42)) (2.1.5)
Requirement already satisfied: mpmath>=0.19 in /opt/conda/lib/python3.11/site-
packages (from sympy->torch>=2.0->-r requirements.txt (line 42)) (1.3.0)
Note: you may need to restart the kernel to use updated packages.

```
[21]: # =====
# IMPORTS
# =====

# -----
# Standard Library Imports
# -----

import pandas as pd
# Data manipulation and tabular metric representation (DataFrame-based
↳reporting)

import os
# Operating system interfaces for environment interaction and path utilities

import time
# Wall-clock time measurement for training and inference performance evaluation

import tempfile
# Temporary file utilities for measuring model memory footprint

from pathlib import Path
# Object-oriented, cross-platform file system path handling

from tqdm.auto import tqdm
# Automatic progress bar rendering for notebooks and terminals (epoch-level
↳training feedback)
```

```

from typing import List, Tuple
# Static type annotations to improve code clarity, readability, and tooling
↳ support

from IPython.display import display, HTML, Markdown
# Rich display of tabular outputs (e.g., Pandas DataFrames) in Jupyter
↳ environments


# -----
# Scientific Computing & Computer Vision Libraries
# -----

import numpy as np
# Numerical computation, array manipulation, and statistical operations

import cv2
# Video decoding, frame preprocessing, color-space conversion, and filtering


# -----
# PyTorch Core Framework
# -----

import torch
# Tensor computation, GPU acceleration, and automatic differentiation

import torch.nn as nn
# Neural network layer definitions and model construction primitives

import torch.optim as optim
# Optimization algorithms for training neural networks

from torch.utils.data import Dataset, DataLoader
# Dataset abstraction and efficient batch-wise data loading


# -----
# Torchvision Utilities
# -----

from torchvision import models, transforms
# Pretrained CNN backbones and image preprocessing pipelines

```

```

# -----
# Evaluation Metrics (Scikit-learn)
# -----

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
# Classification performance metrics and confusion matrix analysis

# -----
# Visualization Libraries
# -----

import matplotlib.pyplot as plt
# Plotting utilities for training curves and comparative analysis

import seaborn as sns
# High-level statistical visualizations and heatmaps

# -----
# Project-Specific Model Architectures
# -----

from models import CNN2DTemporal, CNN3D, EarlyStopping
# Custom deep learning models for video classification

```

```

[11]: # =====
# CONFIGURATION
# =====

torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True

# Current file is inside /code
CODE_DIR = Path.cwd()

# Project root is one level above
PROJECT_ROOT = CODE_DIR.parent

DATASET_ROOT = PROJECT_ROOT / "dataset"
SPLITS_DIR = DATASET_ROOT / "splits"

```

```

LOCAL_WEIGHTS = os.path.join(PROJECT_ROOT, "model", "resnet18-f37072fd.pth")

# Safety checks (VERY IMPORTANT)
assert DATASET_ROOT.exists(), f"Dataset not found at {DATASET_ROOT}"
assert SPLITS_DIR.exists(), f"Splits folder not found at {SPLITS_DIR}"
print("-----")
print(f" Project root: {PROJECT_ROOT}")
print(f" Dataset root: {DATASET_ROOT}")
print(f" Local Weight Path : {LOCAL_WEIGHTS}")
print("-----")
NUM_FRAMES = 16                # Frames sampled per video
IMG_SIZE = (224, 224)          # Required for pretrained CNNs
BATCH_SIZE = 8
EPOCHS = 5
LEARNING_RATE = 1e-4

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f" Running on device: {DEVICE}")
print("-----\n")

```

```

-----
Project root: /home/jovyan/va-
assignment-1/Student_2024ab05275_Video_Classification
Dataset root: /home/jovyan/va-
assignment-1/Student_2024ab05275_Video_Classification/dataset
Local Weight Path : /home/jovyan/va-assignment-
1/Student_2024ab05275_Video_Classification/model/resnet18-f37072fd.pth
-----

Running on device: cuda
-----

```

```

[12]: # =====
# CLASS MAPPING (Derived from dataset folder names)
# =====
# This section automatically discovers class labels from the
# dataset directory structure.
#
# Expected directory format:
# DATASET_ROOT/
#   class_0/
#   class_1/
#   class_2/
#   ...
#
# Each "class_*" folder represents one target class.

```

```

# =====

print("* Scanning dataset root for class folders.....")
print(f"* Dataset root path: {DATASET_ROOT}")
print("-----")
# -----
# Step 1: Discover class directory names
# -----
# - Iterate over all items inside DATASET_ROOT
# - Keep only directories
# - Keep only directory names that start with "class_"
# - Sort them to ensure consistent class index assignment
# -----
CLASS_NAMES = sorted([
    d.name          # Folder name (e.g., "class_1")
    for d in DATASET_ROOT.iterdir()
    if d.is_dir()    # Ensure it is a directory
    and d.name.startswith("class_") # Enforce naming convention
])

print(f"* No of Class Folders Found : {len(CLASS_NAMES)}")
print("* Detected class folders (sorted):")
for cls in CLASS_NAMES:
    print(f"  - {cls}")
print("-----")

# -----
# Step 2: Create class-to-index mapping
# -----
# Assign a unique integer label to each class name.
# The index order is determined by the sorted CLASS_NAMES list.
#
# Example:
#   class_0 -> 0
#   class_1 -> 1
# -----
CLASS_TO_IDX = {cls: idx for idx, cls in enumerate(CLASS_NAMES)}

# -----
# Step 3: Count total number of classes
# -----
NUM_CLASSES = len(CLASS_NAMES)

# -----
# Step 4: Log final class mapping
# -----
print("* Final class-to-index mapping:")

```



```

for class_name, class_idx in CLASS_TO_IDX.items():
    print(f" - {class_name} → {class_idx}")

print("-----")
print(f"* Total number of classes: {NUM_CLASSES}\n")

```

```

* Scanning dataset root for class folders...
* Dataset root path: /home/jovyan/va-
assignment-1/Student_2024ab05275_Video_Classification/dataset
-----

* No of Class Folders Found : 5
* Detected class folders (sorted):
  - class_1_Diving
  - class_2_JumpRope
  - class_3_SkyDiving
  - class_4_Surfing
  - class_5_Swing
-----

* Final class-to-index mapping:
  - class_1_Diving → 0
  - class_2_JumpRope → 1
  - class_3_SkyDiving → 2
  - class_4_Surfing → 3
  - class_5_Swing → 4
-----

* Total number of classes: 5

```

```

[13]: # =====
# VIDEO PREPROCESSING
# =====
"""
- OpenCV used for video loading
- Uniform frame sampling
- Resize to 224×224
- ImageNet normalization (mandatory for pretrained models)
"""

# =====
# IMAGENET PREPROCESSING TRANSFORM
# =====
# This transform prepares raw RGB image frames so they are
# compatible with ImageNet-pretrained CNN backbones
# (e.g., ResNet, EfficientNet).
#
# Expected input:
#   - NumPy array or PIL Image

```

```

#   - Shape: (H, W, C)
#   - Value range: [0, 255]
#
# Output:
#   - torch.Tensor
#   - Shape: (C, H, W)
#   - Normalized using ImageNet statistics
# =====

print("\n Initializing ImageNet preprocessing pipeline...")

# -----
# Step 1: Convert image to PyTorch tensor
# -----
#   - Converts (H, W, C) → (C, H, W)
#   - Scales pixel values from [0, 255] → [0.0, 1.0]
# -----
print(" Adding ToTensor():")
print("   - Converts NumPy/PIL image to torch.Tensor")
print("   - Reorders dimensions to (C, H, W)")
print("   - Scales pixel range to [0.0, 1.0]")

to_tensor_transform = transforms.ToTensor()

# -----
# Step 2: Normalize using ImageNet statistics
# -----
# Normalization ensures that input distribution matches
# what ImageNet-pretrained models were trained on.
#
# Channel order: RGB
#
# Formula per channel:
#   normalized = (x - mean) / std
# -----
print(" Adding Normalize():")
print("   - Mean (RGB): [0.485, 0.456, 0.406]")
print("   - Std (RGB): [0.229, 0.224, 0.225]")

normalize_transform = transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)

# -----
# Step 3: Compose transforms into a single pipeline
# -----

```

```

print(" Composing preprocessing transforms")

imagenet_transform = transforms.Compose([
    to_tensor_transform,
    normalize_transform
])

print(" ImageNet transform pipeline ready")

def load_video(video_path, num_frames=NUM_FRAMES):
    """
    Load a video file, extract frames, apply spatial preprocessing,
    perform uniform temporal sampling, and return a tensor suitable
    for deep learning models.

    Args:
        video_path (Path or str): Path to the video file.
        num_frames (int): Number of frames to sample uniformly.

    Returns:
        torch.Tensor: Video tensor of shape (T, C, H, W)
    """

    # -----
    # Step 1: Open video using OpenCV
    # -----
    cap = cv2.VideoCapture(str(video_path))

    # Sanity check: ensure video file opened correctly
    if not cap.isOpened():
        raise RuntimeError(f" Failed to open video file: {video_path}")

    frames = []
    frame_idx = 0

    # -----
    # Step 2: Read video frame-by-frame
    # -----
    while True:
        ret, frame = cap.read()

        # ret == False indicates end of video or read failure
        if not ret:
            # print(" End of video reached or frame read failed")
            break

```

```

# Convert color space from OpenCV default (BGR) to RGB
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

# Resize frame to match model input size
frame = cv2.resize(frame, IMG_SIZE)

frames.append(frame)
frame_idx += 1

# Periodic logging for long videos
# if frame_idx % 25 == 0:
#     # print(f"    Frames read so far: {frame_idx}")

# -----
# Step 3: Release video capture resource
# -----
cap.release()
# print(f" Total frames extracted: {len(frames)}")

# -----
# Step 4: Uniform temporal sampling
# -----
# Goal: Ensure exactly `num_frames` frames per video
# -----
if len(frames) >= num_frames:
    # print(" Applying uniform temporal sampling")

    # Generate evenly spaced indices across the full video
    idx = np.linspace(
        0,
        len(frames) - 1,
        num_frames
    ).astype(int)

    # print(f" Sampled frame indices: {idx.tolist()}")

    # Select frames at sampled indices
    frames = [frames[i] for i in idx]
else:
    print(" Video shorter than required frames")
    print(" Padding by repeating last frame")

    # Repeat last frame until target length is reached
    while len(frames) < num_frames:
        frames.append(frames[-1])

```

```

# SINGLE-LINE LOG (NEW)
print(
    f" {Path(video_path).name} | "
    f"frames: {frame_idx} → {len(frames)} | "
    f"target: {num_frames} | "
    f"class: {Path(video_path).parent.name}"
)

# -----
# Step 5: Apply ImageNet normalization / transforms
# -----
# print(" Applying ImageNet normalization & transforms")

frames = [imagenet_transform(frame) for frame in frames]

# -----
# Step 6: Stack frames into a single tensor
# -----
# Final shape: (T, C, H, W)
video_tensor = torch.stack(frames)

# print(" Final video tensor shape:", video_tensor.shape)

return video_tensor

```

Initializing ImageNet preprocessing pipeline...

Adding ToTensor():

- Converts NumPy/PIL image to torch.Tensor
- Reorders dimensions to (C, H, W)
- Scales pixel range to [0.0, 1.0]

Adding Normalize():

- Mean (RGB): [0.485, 0.456, 0.406]
- Std (RGB): [0.229, 0.224, 0.225]

Composing preprocessing transforms

ImageNet transform pipeline ready

```

[5]: # =====
# LOAD DATA USING OFFICIAL TRAIN / TEST SPLITS
# =====
# This section loads videos and labels using pre-defined
# split files (e.g., train.txt, test.txt).
#
# Each split file is expected to contain relative paths
# to video files, one per line, such as:
#
# class_0/video_001.avi

```

```

#   class_1/video_023.avi
#
# The parent folder name (class_*) is used as the label.
# =====

def load_split(split_file):
    """
    Load videos and labels from a split file.

    Args:
        split_file (Path): Path to the split text file.

    Returns:
        Tuple[torch.Tensor, torch.Tensor]:
            - videos: Tensor of shape (N, T, C, H, W)
            - labels: Tensor of shape (N,)
    """

    print("\n =====")
    print(" Loading split file")
    print(f" Split file path: {split_file}")
    print(" =====")

    videos = [] # Will store per-video tensors
    labels = [] # Will store integer class labels

    # -----
    # Step 1: Read split file
    # -----
    with open(split_file, "r") as f:
        lines = f.read().splitlines()

    print(f" Total entries found in split file: {len(lines)}")

    # -----
    # Step 2: Process each video entry
    # -----
    for idx, line in enumerate(lines):
        # Construct absolute video path
        video_path = DATASET_ROOT / line

        # Extract class name from path
        # Example: "class_2/video_003.avi" → "class_2"
        class_name = line.split("/") [0]

        # Map class name to integer label
        label = CLASS_TO_IDX[class_name]

```

```

        """
        print("\n -----")
        print(f" Processing video {idx + 1}/{len(lines)}")
        print(f" Relative path : {line}")
        print(f" Absolute path : {video_path}")
        print(f" Class name : {class_name}")
        print(f" Class index : {label}")
        print(" -----")
        """

        # Load and preprocess video (T, C, H, W)
        video_tensor = load_video(video_path)

        # Append video tensor and label
        videos.append(video_tensor)
        labels.append(label)

    # -----
    # Step 3: Stack videos and labels into tensors
    # -----
    print("\n Stacking videos and labels into tensors")

    videos_tensor = torch.stack(videos)          # Shape: (N, T, C, H, W)
    labels_tensor = torch.tensor(labels)         # Shape: (N,)

    print(" Split loaded successfully !! ")
    print(f" Videos tensor shape: {videos_tensor.shape}")
    print(f" Labels tensor shape: {labels_tensor.shape}")
    print(" =====\n")

    return videos_tensor, labels_tensor

# =====
# LOAD TRAINING DATA
# =====
print("\n-----")
print(" Loading TRAINING data")
print("-----\n")

X_train, y_train = load_split(SPLITS_DIR / "train.txt")

# =====
# LOAD TESTING DATA
# =====
print("\n-----")
print(" Loading TESTING data")

```

```

print("-----\n")

X_test, y_test = load_split(SPLITS_DIR / "test.txt")

# =====
# LOAD VALIDATION DATA
# =====
print("\n-----")
print(" Loading VALIDATION data")
print("-----\n")

X_val, y_val = load_split(SPLITS_DIR / "val.txt")

# =====
# DATASET SUMMARY
# =====
print("\n-----")
print(" Dataset summary")
print("-----\n")

print(f" Training videos      : {len(X_train)}")
print(f" Testing videos       : {len(X_test)}")
print(f" Validation videos     : {len(X_val)}")
print(f" Total classes        : {NUM_CLASSES}")
print(f" Train tensor          : {X_train.shape} (N, T, C, H, W)")
print(f" Test tensor           : {X_test.shape}  (N, T, C, H, W)")
print(f" Validation tensor     : {X_val.shape}   (N, T, C, H, W)")

```

Loading TRAINING data

```

=====
Loading split file
Split file path: /home/jovyan/va-
assignment-1/Student_2024ab05275_Video_Classification/dataset/splits/train.txt
=====
Total entries found in split file: 280
v_Diving_g12_c02.avi | frames: 158 → 16 | target: 16 | class: class_1_Diving
v_Diving_g09_c04.avi | frames: 176 → 16 | target: 16 | class: class_1_Diving
v_Diving_g10_c05.avi | frames: 163 → 16 | target: 16 | class: class_1_Diving
v_Diving_g19_c02.avi | frames: 171 → 16 | target: 16 | class: class_1_Diving
v_Diving_g14_c04.avi | frames: 203 → 16 | target: 16 | class: class_1_Diving
v_Diving_g15_c06.avi | frames: 195 → 16 | target: 16 | class: class_1_Diving
v_Diving_g17_c02.avi | frames: 153 → 16 | target: 16 | class: class_1_Diving

```


v_Diving_g02_c05.avi	frames: 200 → 16	target: 16	class: class_1_Diving
v_Diving_g16_c05.avi	frames: 247 → 16	target: 16	class: class_1_Diving
v_Diving_g06_c03.avi	frames: 236 → 16	target: 16	class: class_1_Diving
v_Diving_g18_c03.avi	frames: 154 → 16	target: 16	class: class_1_Diving
v_Diving_g09_c03.avi	frames: 212 → 16	target: 16	class: class_1_Diving
v_Diving_g22_c06.avi	frames: 179 → 16	target: 16	class: class_1_Diving
v_Diving_g06_c01.avi	frames: 212 → 16	target: 16	class: class_1_Diving
v_Diving_g03_c04.avi	frames: 197 → 16	target: 16	class: class_1_Diving
v_Diving_g02_c04.avi	frames: 160 → 16	target: 16	class: class_1_Diving
v_Diving_g12_c04.avi	frames: 202 → 16	target: 16	class: class_1_Diving
v_Diving_g16_c01.avi	frames: 310 → 16	target: 16	class: class_1_Diving
v_Diving_g15_c02.avi	frames: 201 → 16	target: 16	class: class_1_Diving
v_Diving_g19_c04.avi	frames: 207 → 16	target: 16	class: class_1_Diving
v_Diving_g04_c05.avi	frames: 150 → 16	target: 16	class: class_1_Diving
v_Diving_g15_c01.avi	frames: 176 → 16	target: 16	class: class_1_Diving
v_Diving_g13_c07.avi	frames: 182 → 16	target: 16	class: class_1_Diving
v_Diving_g21_c03.avi	frames: 263 → 16	target: 16	class: class_1_Diving
v_Diving_g02_c06.avi	frames: 152 → 16	target: 16	class: class_1_Diving
v_Diving_g01_c05.avi	frames: 234 → 16	target: 16	class: class_1_Diving
v_Diving_g12_c03.avi	frames: 191 → 16	target: 16	class: class_1_Diving
v_Diving_g24_c06.avi	frames: 170 → 16	target: 16	class: class_1_Diving
v_Diving_g21_c02.avi	frames: 187 → 16	target: 16	class: class_1_Diving
v_Diving_g12_c07.avi	frames: 213 → 16	target: 16	class: class_1_Diving
v_Diving_g13_c02.avi	frames: 153 → 16	target: 16	class: class_1_Diving
v_Diving_g21_c04.avi	frames: 322 → 16	target: 16	class: class_1_Diving
v_Diving_g23_c04.avi	frames: 202 → 16	target: 16	class: class_1_Diving
v_Diving_g17_c01.avi	frames: 301 → 16	target: 16	class: class_1_Diving
v_Diving_g06_c06.avi	frames: 219 → 16	target: 16	class: class_1_Diving
v_Diving_g16_c03.avi	frames: 204 → 16	target: 16	class: class_1_Diving
v_Diving_g07_c03.avi	frames: 216 → 16	target: 16	class: class_1_Diving
v_Diving_g03_c02.avi	frames: 192 → 16	target: 16	class: class_1_Diving
v_Diving_g16_c02.avi	frames: 172 → 16	target: 16	class: class_1_Diving
v_Diving_g09_c02.avi	frames: 178 → 16	target: 16	class: class_1_Diving
v_Diving_g08_c05.avi	frames: 155 → 16	target: 16	class: class_1_Diving
v_Diving_g07_c04.avi	frames: 173 → 16	target: 16	class: class_1_Diving
v_Diving_g21_c06.avi	frames: 192 → 16	target: 16	class: class_1_Diving
v_Diving_g10_c06.avi	frames: 166 → 16	target: 16	class: class_1_Diving
v_Diving_g02_c02.avi	frames: 151 → 16	target: 16	class: class_1_Diving
v_Diving_g18_c01.avi	frames: 256 → 16	target: 16	class: class_1_Diving
v_Diving_g12_c06.avi	frames: 308 → 16	target: 16	class: class_1_Diving
v_Diving_g15_c05.avi	frames: 210 → 16	target: 16	class: class_1_Diving
v_Diving_g01_c07.avi	frames: 243 → 16	target: 16	class: class_1_Diving
v_Diving_g08_c03.avi	frames: 158 → 16	target: 16	class: class_1_Diving
v_Diving_g13_c01.avi	frames: 226 → 16	target: 16	class: class_1_Diving
v_Diving_g12_c01.avi	frames: 161 → 16	target: 16	class: class_1_Diving
v_Diving_g03_c01.avi	frames: 190 → 16	target: 16	class: class_1_Diving
v_Diving_g23_c06.avi	frames: 383 → 16	target: 16	class: class_1_Diving
v_Diving_g01_c02.avi	frames: 282 → 16	target: 16	class: class_1_Diving

v_Diving_g07_c02.avi | frames: 158 → 16 | target: 16 | class: class_1_Diving
 v_SkyDiving_g07_c01.avi | frames: 167 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g03_c04.avi | frames: 232 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g22_c03.avi | frames: 298 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g16_c01.avi | frames: 198 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g05_c01.avi | frames: 186 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g24_c01.avi | frames: 135 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g07_c03.avi | frames: 203 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g22_c04.avi | frames: 304 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g09_c03.avi | frames: 203 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g20_c06.avi | frames: 248 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g18_c04.avi | frames: 176 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g20_c02.avi | frames: 198 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g18_c02.avi | frames: 265 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g04_c01.avi | frames: 149 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g19_c03.avi | frames: 167 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g03_c01.avi | frames: 187 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g08_c02.avi | frames: 244 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g13_c03.avi | frames: 196 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g11_c02.avi | frames: 200 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g04_c02.avi | frames: 141 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g10_c03.avi | frames: 189 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g10_c01.avi | frames: 196 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g21_c03.avi | frames: 203 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g05_c02.avi | frames: 170 → 16 | target: 16 | class:

```

class_3_SkyDiving
  v_SkyDiving_g24_c04.avi | frames: 286 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g20_c01.avi | frames: 136 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g23_c02.avi | frames: 182 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g02_c02.avi | frames: 127 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g14_c04.avi | frames: 167 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g05_c03.avi | frames: 200 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g17_c03.avi | frames: 200 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g08_c01.avi | frames: 180 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g22_c01.avi | frames: 329 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g04_c03.avi | frames: 127 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g20_c04.avi | frames: 248 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g22_c02.avi | frames: 322 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g23_c01.avi | frames: 166 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g25_c03.avi | frames: 192 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g06_c04.avi | frames: 154 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g25_c04.avi | frames: 201 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g14_c01.avi | frames: 210 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g17_c05.avi | frames: 182 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g15_c02.avi | frames: 172 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g17_c02.avi | frames: 181 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g12_c02.avi | frames: 150 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g07_c02.avi | frames: 179 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g18_c05.avi | frames: 224 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g09_c04.avi | frames: 215 → 16 | target: 16 | class:

```

```

class_3_SkyDiving
  v_SkyDiving_g01_c02.avi | frames: 194 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g24_c02.avi | frames: 229 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g01_c03.avi | frames: 126 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g16_c02.avi | frames: 213 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g08_c04.avi | frames: 211 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g02_c03.avi | frames: 136 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g21_c04.avi | frames: 209 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g17_c04.avi | frames: 208 → 16 | target: 16 | class:
class_3_SkyDiving
  v_Surfing_g15_c05.avi | frames: 142 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g16_c02.avi | frames: 239 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g17_c03.avi | frames: 251 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g10_c04.avi | frames: 266 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g10_c03.avi | frames: 126 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g19_c02.avi | frames: 232 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g20_c03.avi | frames: 164 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g11_c02.avi | frames: 255 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g01_c01.avi | frames: 254 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g06_c02.avi | frames: 265 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g05_c03.avi | frames: 277 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g02_c04.avi | frames: 243 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g16_c01.avi | frames: 259 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g14_c04.avi | frames: 211 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g07_c01.avi | frames: 264 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g08_c06.avi | frames: 249 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g19_c01.avi | frames: 330 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g20_c06.avi | frames: 259 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g03_c01.avi | frames: 125 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g15_c07.avi | frames: 158 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g15_c06.avi | frames: 178 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g12_c03.avi | frames: 210 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g07_c02.avi | frames: 241 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g15_c02.avi | frames: 163 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g17_c06.avi | frames: 246 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g04_c04.avi | frames: 132 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g25_c02.avi | frames: 153 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g16_c06.avi | frames: 248 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g06_c03.avi | frames: 125 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g01_c02.avi | frames: 203 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g12_c01.avi | frames: 364 → 16 | target: 16 | class: class_4_Surfing

```

v_Surfing_g24_c02.avi	frames: 247 → 16	target: 16	class: class_4_Surfing
v_Surfing_g15_c01.avi	frames: 155 → 16	target: 16	class: class_4_Surfing
v_Surfing_g11_c01.avi	frames: 196 → 16	target: 16	class: class_4_Surfing
v_Surfing_g08_c04.avi	frames: 237 → 16	target: 16	class: class_4_Surfing
v_Surfing_g05_c04.avi	frames: 241 → 16	target: 16	class: class_4_Surfing
v_Surfing_g12_c06.avi	frames: 168 → 16	target: 16	class: class_4_Surfing
v_Surfing_g17_c01.avi	frames: 232 → 16	target: 16	class: class_4_Surfing
v_Surfing_g16_c07.avi	frames: 236 → 16	target: 16	class: class_4_Surfing
v_Surfing_g12_c04.avi	frames: 226 → 16	target: 16	class: class_4_Surfing
v_Surfing_g20_c02.avi	frames: 243 → 16	target: 16	class: class_4_Surfing
v_Surfing_g17_c02.avi	frames: 250 → 16	target: 16	class: class_4_Surfing
v_Surfing_g09_c05.avi	frames: 255 → 16	target: 16	class: class_4_Surfing
v_Surfing_g21_c02.avi	frames: 186 → 16	target: 16	class: class_4_Surfing
v_Surfing_g21_c04.avi	frames: 152 → 16	target: 16	class: class_4_Surfing
v_Surfing_g13_c04.avi	frames: 126 → 16	target: 16	class: class_4_Surfing
v_Surfing_g02_c01.avi	frames: 158 → 16	target: 16	class: class_4_Surfing
v_Surfing_g21_c01.avi	frames: 130 → 16	target: 16	class: class_4_Surfing
v_Surfing_g19_c04.avi	frames: 266 → 16	target: 16	class: class_4_Surfing
v_Surfing_g08_c01.avi	frames: 192 → 16	target: 16	class: class_4_Surfing
v_Surfing_g09_c03.avi	frames: 219 → 16	target: 16	class: class_4_Surfing
v_Surfing_g21_c03.avi	frames: 137 → 16	target: 16	class: class_4_Surfing
v_Surfing_g01_c03.avi	frames: 220 → 16	target: 16	class: class_4_Surfing
v_Surfing_g10_c05.avi	frames: 253 → 16	target: 16	class: class_4_Surfing
v_Surfing_g10_c06.avi	frames: 191 → 16	target: 16	class: class_4_Surfing
v_Surfing_g20_c05.avi	frames: 246 → 16	target: 16	class: class_4_Surfing
v_Swing_g17_c02.avi	frames: 168 → 16	target: 16	class: class_5_Swing
v_Swing_g01_c05.avi	frames: 201 → 16	target: 16	class: class_5_Swing
v_Swing_g02_c05.avi	frames: 201 → 16	target: 16	class: class_5_Swing
v_Swing_g03_c02.avi	frames: 145 → 16	target: 16	class: class_5_Swing
v_Swing_g08_c02.avi	frames: 160 → 16	target: 16	class: class_5_Swing
v_Swing_g07_c05.avi	frames: 151 → 16	target: 16	class: class_5_Swing
v_Swing_g19_c02.avi	frames: 276 → 16	target: 16	class: class_5_Swing
v_Swing_g09_c06.avi	frames: 140 → 16	target: 16	class: class_5_Swing
v_Swing_g12_c03.avi	frames: 126 → 16	target: 16	class: class_5_Swing
v_Swing_g21_c02.avi	frames: 168 → 16	target: 16	class: class_5_Swing
v_Swing_g22_c02.avi	frames: 201 → 16	target: 16	class: class_5_Swing
v_Swing_g17_c01.avi	frames: 160 → 16	target: 16	class: class_5_Swing
v_Swing_g11_c06.avi	frames: 251 → 16	target: 16	class: class_5_Swing
v_Swing_g15_c05.avi	frames: 201 → 16	target: 16	class: class_5_Swing
v_Swing_g13_c04.avi	frames: 160 → 16	target: 16	class: class_5_Swing
v_Swing_g09_c05.avi	frames: 151 → 16	target: 16	class: class_5_Swing
v_Swing_g14_c04.avi	frames: 151 → 16	target: 16	class: class_5_Swing
v_Swing_g06_c02.avi	frames: 126 → 16	target: 16	class: class_5_Swing
v_Swing_g09_c03.avi	frames: 126 → 16	target: 16	class: class_5_Swing
v_Swing_g14_c03.avi	frames: 151 → 16	target: 16	class: class_5_Swing
v_Swing_g07_c02.avi	frames: 251 → 16	target: 16	class: class_5_Swing
v_Swing_g09_c02.avi	frames: 126 → 16	target: 16	class: class_5_Swing
v_Swing_g09_c01.avi	frames: 126 → 16	target: 16	class: class_5_Swing

v_Swing_g11_c05.avi | frames: 151 → 16 | target: 16 | class: class_5_Swing
v_Swing_g18_c04.avi | frames: 248 → 16 | target: 16 | class: class_5_Swing
v_Swing_g16_c05.avi | frames: 151 → 16 | target: 16 | class: class_5_Swing
v_Swing_g03_c04.avi | frames: 145 → 16 | target: 16 | class: class_5_Swing
v_Swing_g19_c01.avi | frames: 276 → 16 | target: 16 | class: class_5_Swing
v_Swing_g12_c04.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g23_c05.avi | frames: 151 → 16 | target: 16 | class: class_5_Swing
v_Swing_g18_c01.avi | frames: 335 → 16 | target: 16 | class: class_5_Swing
v_Swing_g21_c05.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
v_Swing_g09_c04.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g11_c04.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g18_c02.avi | frames: 335 → 16 | target: 16 | class: class_5_Swing
v_Swing_g23_c01.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g20_c06.avi | frames: 151 → 16 | target: 16 | class: class_5_Swing
v_Swing_g08_c05.avi | frames: 169 → 16 | target: 16 | class: class_5_Swing
v_Swing_g12_c01.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g23_c03.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g22_c03.avi | frames: 191 → 16 | target: 16 | class: class_5_Swing
v_Swing_g17_c04.avi | frames: 160 → 16 | target: 16 | class: class_5_Swing
v_Swing_g02_c03.avi | frames: 300 → 16 | target: 16 | class: class_5_Swing
v_Swing_g11_c01.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g11_c02.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g20_c05.avi | frames: 151 → 16 | target: 16 | class: class_5_Swing
v_Swing_g03_c03.avi | frames: 145 → 16 | target: 16 | class: class_5_Swing
v_Swing_g24_c04.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g03_c01.avi | frames: 145 → 16 | target: 16 | class: class_5_Swing
v_Swing_g04_c06.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
v_Swing_g04_c02.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g06_c01.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
v_Swing_g05_c06.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
v_Swing_g10_c02.avi | frames: 251 → 16 | target: 16 | class: class_5_Swing
v_Swing_g15_c07.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
v_Swing_g07_c06.avi | frames: 161 → 16 | target: 16 | class: class_5_Swing
v_JumpRope_g13_c04.avi | frames: 374 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g13_c01.avi | frames: 539 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g14_c01.avi | frames: 296 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g17_c03.avi | frames: 586 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g24_c01.avi | frames: 410 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g12_c04.avi | frames: 398 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g12_c02.avi | frames: 403 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g24_c06.avi | frames: 384 → 16 | target: 16 | class:

```

class_2_JumpRope
  v_JumpRope_g22_c01.avi | frames: 198 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g16_c06.avi | frames: 529 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g10_c01.avi | frames: 598 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g17_c02.avi | frames: 599 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g20_c03.avi | frames: 509 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g05_c03.avi | frames: 617 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g23_c05.avi | frames: 242 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g11_c02.avi | frames: 533 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g25_c05.avi | frames: 391 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g24_c03.avi | frames: 411 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g25_c06.avi | frames: 408 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g15_c01.avi | frames: 548 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g04_c01.avi | frames: 498 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g14_c04.avi | frames: 306 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g20_c04.avi | frames: 510 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g24_c05.avi | frames: 358 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g11_c07.avi | frames: 496 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g08_c02.avi | frames: 443 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g16_c07.avi | frames: 464 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g07_c06.avi | frames: 403 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g03_c01.avi | frames: 294 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g19_c03.avi | frames: 335 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g15_c05.avi | frames: 531 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g07_c05.avi | frames: 343 → 16 | target: 16 | class:

```

```

class_2_JumpRope
  v_JumpRope_g16_c05.avi | frames: 485 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g17_c01.avi | frames: 614 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g01_c01.avi | frames: 257 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g21_c02.avi | frames: 445 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g13_c03.avi | frames: 482 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g15_c02.avi | frames: 352 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g11_c06.avi | frames: 510 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g21_c06.avi | frames: 490 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g02_c01.avi | frames: 448 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g04_c04.avi | frames: 546 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g04_c06.avi | frames: 537 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g09_c01.avi | frames: 472 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g14_c02.avi | frames: 305 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g15_c04.avi | frames: 489 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g03_c03.avi | frames: 294 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g23_c01.avi | frames: 343 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g20_c01.avi | frames: 458 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g06_c03.avi | frames: 355 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g17_c05.avi | frames: 575 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g03_c02.avi | frames: 305 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g09_c05.avi | frames: 510 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g24_c02.avi | frames: 391 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g09_c03.avi | frames: 500 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g12_c01.avi | frames: 374 → 16 | target: 16 | class:

```


class_2_JumpRope

```
Stacking videos and labels into tensors
Split loaded successfully !!
Videos tensor shape: torch.Size([280, 16, 3, 224, 224])
Labels tensor shape: torch.Size([280])
=====
```

```
-----
Loading TESTING data
-----
```

```
=====
Loading split file
Split file path: /home/jovyan/va-
assignment-1/Student_2024ab05275_Video_Classification/dataset/splits/test.txt
=====
Total entries found in split file: 60
v_Diving_g04_c04.avi | frames: 285 → 16 | target: 16 | class: class_1_Diving
v_Diving_g14_c01.avi | frames: 218 → 16 | target: 16 | class: class_1_Diving
v_Diving_g13_c03.avi | frames: 171 → 16 | target: 16 | class: class_1_Diving
v_Diving_g24_c04.avi | frames: 162 → 16 | target: 16 | class: class_1_Diving
v_Diving_g09_c01.avi | frames: 192 → 16 | target: 16 | class: class_1_Diving
v_Diving_g17_c03.avi | frames: 153 → 16 | target: 16 | class: class_1_Diving
v_Diving_g06_c04.avi | frames: 216 → 16 | target: 16 | class: class_1_Diving
v_Diving_g08_c06.avi | frames: 219 → 16 | target: 16 | class: class_1_Diving
v_Diving_g19_c01.avi | frames: 207 → 16 | target: 16 | class: class_1_Diving
v_Diving_g22_c04.avi | frames: 220 → 16 | target: 16 | class: class_1_Diving
v_Diving_g03_c05.avi | frames: 248 → 16 | target: 16 | class: class_1_Diving
v_Diving_g13_c05.avi | frames: 162 → 16 | target: 16 | class: class_1_Diving
v_SkyDiving_g25_c02.avi | frames: 180 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g05_c05.avi | frames: 220 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g14_c02.avi | frames: 198 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g12_c04.avi | frames: 356 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g11_c01.avi | frames: 209 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g06_c03.avi | frames: 160 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g09_c02.avi | frames: 208 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g13_c02.avi | frames: 216 → 16 | target: 16 | class:
class_3_SkyDiving
```

v_SkyDiving_g09_c01.avi | frames: 198 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g01_c01.avi | frames: 187 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g03_c05.avi | frames: 247 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_SkyDiving_g25_c01.avi | frames: 133 → 16 | target: 16 | class:
 class_3_SkyDiving
 v_Surfing_g24_c03.avi | frames: 245 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g13_c05.avi | frames: 249 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g14_c02.avi | frames: 205 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g17_c05.avi | frames: 193 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g17_c07.avi | frames: 229 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g19_c03.avi | frames: 174 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g24_c04.avi | frames: 250 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g04_c03.avi | frames: 144 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g08_c03.avi | frames: 337 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g09_c01.avi | frames: 206 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g10_c01.avi | frames: 244 → 16 | target: 16 | class: class_4_Surfing
 v_Surfing_g11_c04.avi | frames: 178 → 16 | target: 16 | class: class_4_Surfing
 v_Swing_g23_c04.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g24_c05.avi | frames: 151 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g06_c04.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g16_c01.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g12_c02.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g14_c02.avi | frames: 151 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g07_c03.avi | frames: 251 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g24_c01.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g04_c07.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g13_c02.avi | frames: 160 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g07_c04.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
 v_Swing_g25_c01.avi | frames: 318 → 16 | target: 16 | class: class_5_Swing
 v_JumpRope_g21_c04.avi | frames: 503 → 16 | target: 16 | class:
 class_2_JumpRope
 v_JumpRope_g19_c01.avi | frames: 307 → 16 | target: 16 | class:
 class_2_JumpRope
 v_JumpRope_g12_c03.avi | frames: 398 → 16 | target: 16 | class:
 class_2_JumpRope
 v_JumpRope_g08_c03.avi | frames: 526 → 16 | target: 16 | class:
 class_2_JumpRope
 v_JumpRope_g04_c07.avi | frames: 518 → 16 | target: 16 | class:
 class_2_JumpRope
 v_JumpRope_g12_c05.avi | frames: 403 → 16 | target: 16 | class:
 class_2_JumpRope
 v_JumpRope_g07_c03.avi | frames: 388 → 16 | target: 16 | class:
 class_2_JumpRope
 v_JumpRope_g21_c05.avi | frames: 502 → 16 | target: 16 | class:
 class_2_JumpRope

```

v_JumpRope_g05_c05.avi | frames: 623 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g22_c06.avi | frames: 221 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g16_c03.avi | frames: 507 → 16 | target: 16 | class:
class_2_JumpRope
v_JumpRope_g20_c02.avi | frames: 527 → 16 | target: 16 | class:
class_2_JumpRope

```

```

Stacking videos and labels into tensors
Split loaded successfully !!
Videos tensor shape: torch.Size([60, 16, 3, 224, 224])
Labels tensor shape: torch.Size([60])
=====

```

```

-----
Loading VALIDATION data
-----

```

```

=====
Loading split file
Split file path: /home/jovyan/va-
assignment-1/Student_2024ab05275_Video_Classification/dataset/splits/val.txt
=====
Total entries found in split file: 60
v_Diving_g07_c01.avi | frames: 179 → 16 | target: 16 | class: class_1_Diving
v_Diving_g22_c05.avi | frames: 162 → 16 | target: 16 | class: class_1_Diving
v_Diving_g14_c02.avi | frames: 304 → 16 | target: 16 | class: class_1_Diving
v_Diving_g21_c05.avi | frames: 181 → 16 | target: 16 | class: class_1_Diving
v_Diving_g06_c02.avi | frames: 209 → 16 | target: 16 | class: class_1_Diving
v_Diving_g03_c03.avi | frames: 187 → 16 | target: 16 | class: class_1_Diving
v_Diving_g01_c04.avi | frames: 232 → 16 | target: 16 | class: class_1_Diving
v_Diving_g11_c05.avi | frames: 170 → 16 | target: 16 | class: class_1_Diving
v_Diving_g01_c06.avi | frames: 255 → 16 | target: 16 | class: class_1_Diving
v_Diving_g15_c07.avi | frames: 213 → 16 | target: 16 | class: class_1_Diving
v_Diving_g21_c07.avi | frames: 182 → 16 | target: 16 | class: class_1_Diving
v_Diving_g11_c06.avi | frames: 181 → 16 | target: 16 | class: class_1_Diving
v_SkyDiving_g14_c03.avi | frames: 201 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g18_c01.avi | frames: 175 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g11_c04.avi | frames: 203 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g23_c04.avi | frames: 167 → 16 | target: 16 | class:
class_3_SkyDiving
v_SkyDiving_g05_c04.avi | frames: 211 → 16 | target: 16 | class:

```

```

class_3_SkyDiving
  v_SkyDiving_g15_c04.avi | frames: 222 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g11_c03.avi | frames: 212 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g16_c03.avi | frames: 200 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g06_c02.avi | frames: 153 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g07_c05.avi | frames: 156 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g18_c03.avi | frames: 173 → 16 | target: 16 | class:
class_3_SkyDiving
  v_SkyDiving_g21_c01.avi | frames: 209 → 16 | target: 16 | class:
class_3_SkyDiving
  v_Surfing_g01_c07.avi | frames: 235 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g25_c03.avi | frames: 128 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g23_c04.avi | frames: 186 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g01_c04.avi | frames: 283 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g13_c01.avi | frames: 204 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g07_c03.avi | frames: 196 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g16_c04.avi | frames: 248 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g13_c02.avi | frames: 246 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g20_c04.avi | frames: 257 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g18_c01.avi | frames: 125 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g03_c03.avi | frames: 125 → 16 | target: 16 | class: class_4_Surfing
  v_Surfing_g17_c04.avi | frames: 172 → 16 | target: 16 | class: class_4_Surfing
  v_Swing_g05_c07.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g21_c01.avi | frames: 168 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g25_c02.avi | frames: 318 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g19_c03.avi | frames: 276 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g02_c01.avi | frames: 135 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g08_c03.avi | frames: 160 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g10_c04.avi | frames: 231 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g17_c03.avi | frames: 160 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g05_c05.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g23_c02.avi | frames: 126 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g25_c04.avi | frames: 317 → 16 | target: 16 | class: class_5_Swing
  v_Swing_g15_c06.avi | frames: 201 → 16 | target: 16 | class: class_5_Swing
  v_JumpRope_g09_c06.avi | frames: 491 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g02_c06.avi | frames: 500 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g09_c07.avi | frames: 491 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g01_c02.avi | frames: 268 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g19_c04.avi | frames: 476 → 16 | target: 16 | class:

```

```

class_2_JumpRope
  v_JumpRope_g09_c02.avi | frames: 501 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g18_c07.avi | frames: 405 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g22_c02.avi | frames: 209 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g10_c04.avi | frames: 570 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g05_c01.avi | frames: 621 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g18_c04.avi | frames: 382 → 16 | target: 16 | class:
class_2_JumpRope
  v_JumpRope_g08_c01.avi | frames: 497 → 16 | target: 16 | class:
class_2_JumpRope

```

```

Stacking videos and labels into tensors
Split loaded successfully !!
Videos tensor shape: torch.Size([60, 16, 3, 224, 224])
Labels tensor shape: torch.Size([60])
=====

```

```

-----
Dataset summary
-----

```

```

Training videos      : 280
Testing videos       : 60
Validation videos    : 60
Total classes        : 5
Train tensor         : torch.Size([280, 16, 3, 224, 224]) (N, T, C, H, W)
Test tensor          : torch.Size([60, 16, 3, 224, 224]) (N, T, C, H, W)
Validation tensor     : torch.Size([60, 16, 3, 224, 224]) (N, T, C, H, W)

```

```

[6]: # =====
# VIDEO DATASET WITH OPTIONAL DATA AUGMENTATION
# =====
class VideoDataset(Dataset):
    """
    Custom PyTorch Dataset for video classification.

    Each sample consists of:
    - A video: sequence of frames (Tensor)
    - A label: class index or class name

    Data augmentation:
    """

```

```

- Random horizontal flip is applied
  ONLY when train=True
"""

def __init__(
    self,
    X: List[torch.Tensor],
    y: List[int],
    train: bool = True,
) -> None:
    """
    Initialize the dataset.

    Args:
        X (List[Tensor]):
            List of videos.
            Each video is a Tensor of shape:
            (num_frames, channels, height, width)

        y (List[int]):
            Corresponding labels for each video.

        train (bool):
            If True:
                - Apply data augmentation (horizontal flip)
            If False:
                - No augmentation (used for validation/testing)
    """
    # Store videos
    self.X = X

    # Store labels
    self.y = y

    # Flag to control augmentation behavior
    self.train = train

    # Define spatial augmentation:
    # Randomly flips an image horizontally with 50% probability
    #
    # IMPORTANT:
    # - This does NOT add new pixels
    # - It only rearranges existing pixels
    self.flip = transforms.RandomHorizontalFlip(p=0.5)

def __len__(self) -> int:
    """

```

```

        Return the total number of samples in the dataset.

        This method is required by PyTorch's Dataset class
        so that DataLoader knows how many samples exist.
        """
        return len(self.X)

def __getitem__(self, idx: int) -> Tuple[torch.Tensor, int]:
    """
    Retrieve one sample from the dataset.

    Args:
        idx (int):
            Index of the sample to retrieve.

    Returns:
        Tuple[Tensor, int]:
            - video: Tensor of shape
              (num_frames, channels, height, width)
            - label: corresponding class label
    """
    # Fetch the video at the given index
    video = self.X[idx]

    # Apply data augmentation ONLY during training
    if self.train:
        # Apply horizontal flip independently to each frame
        #
        # Why per-frame?
        # - Each frame is treated as an image
        # - Maintains temporal order
        # - Simple and effective spatial augmentation
        #
        # torch.stack is used to reconstruct the video
        # back into a single Tensor
        video = torch.stack([self.flip(frame) for frame in video])

    # Return video and its label
    return video, self.y[idx]

```

```

[7]: # =====
# DATALOADERS
# =====
train_loader = DataLoader(
    VideoDataset(X_train, y_train, train=True),
    batch_size=BATCH_SIZE, shuffle=True
)

```

```

print("\n Training - Data loaders initialized successfully !!")

test_loader = DataLoader(
    VideoDataset(X_test, y_test, train=False),
    batch_size=BATCH_SIZE, shuffle=False
)

print("\n Testing - Data loaders initialized successfully !!")

val_loader = DataLoader(
    VideoDataset(X_val, y_val, train=False),
    batch_size=BATCH_SIZE, shuffle=False
)

print("\n Validation - Data loaders initialized successfully !!")

```

Training - Data loaders initialized successfully !!

Testing - Data loaders initialized successfully !!

Validation - Data loaders initialized successfully !!

```

[17]: # =====
# TRAINING & EVALUATION FUNCTION (PDF-SAFE)
# =====
def train_and_evaluate(model, model_name):
    """
    Train a deep learning video classification model and evaluate
    its performance using validation-based early stopping.

    Outputs:
    - Training loss curve (saved + displayed)
    - Confusion matrix (saved + displayed)
    - Classification report (tabulated)
    - Tabulated performance metrics (Pandas DataFrame)
    - Model parameters & memory usage

    Args:
        model (torch.nn.Module): Model to be trained
        model_name (str): Descriptive model name

    Returns:
        pd.DataFrame: Tabulated evaluation metrics
    """

```



```

# -----
# Directory structure for storing results
# -----
base_dir = PROJECT_ROOT / "results"
cm_dir = base_dir / "confusion_matrices"
perf_dir = base_dir / "performance_plots"
feat_dir = base_dir / "feature_visualizations"

for directory in [cm_dir, perf_dir, feat_dir]:
    directory.mkdir(parents=True, exist_ok=True)

# -----
# Model & optimization setup
# -----
print("\n-----")
print(f" Training model: {model_name}")
print("-----\n")

model.to(DEVICE)

optimizer = optim.Adam(
    model.parameters(),
    lr=LEARNING_RATE,
    weight_decay=1e-4 # L2 regularization
)

criterion = nn.CrossEntropyLoss()

scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=EPOCHS
)

# -----
# Early stopping utility
# -----
class EarlyStopping:
    """
    Stops training if validation loss does not improve
    for a fixed number of epochs (patience).
    """

    def __init__(self, patience=5, min_delta=1e-4):
        self.patience = patience
        self.min_delta = min_delta
        self.best_loss = float("inf")
        self.counter = 0

```

```

        self.should_stop = False

    def step(self, val_loss):
        if val_loss < self.best_loss - self.min_delta:
            self.best_loss = val_loss
            self.counter = 0
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.should_stop = True

early_stopper = EarlyStopping(patience=5)

# -----
# Training loop with simple epoch prints
# -----
epoch_losses = []
start_train_time = time.time()

for epoch in range(EPOCHS):

    # ----- TRAIN -----
    model.train()
    running_loss = 0.0

    for videos, labels in train_loader:
        videos = videos.to(DEVICE)
        labels = labels.to(DEVICE)

        optimizer.zero_grad()
        outputs = model(videos)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    train_loss = running_loss / len(train_loader)
    epoch_losses.append(train_loss)

    # ----- VALIDATION -----
    model.eval()
    val_loss = 0.0

    with torch.no_grad():
        for videos, labels in val_loader:
            videos = videos.to(DEVICE)

```

```

        labels = labels.to(DEVICE)

        outputs = model(videos)
        loss = criterion(outputs, labels)
        val_loss += loss.item()

    val_loss /= len(val_loader)

    # ----- Print epoch summary -----
    print(f"Epoch [{epoch+1}/{EPOCHS}] | Train Loss: {train_loss:.4f} | Val_
↪Loss: {val_loss:.4f}")

    # ----- Early stopping -----
    early_stopper.step(val_loss)
    if early_stopper.should_stop:
        print(f" Early stopping triggered at epoch {epoch + 1}")
        break

    scheduler.step()

train_time = time.time() - start_train_time

# -----
# Training loss curve (SAVE + DISPLAY)
# -----
plt.figure()
plt.plot(epoch_losses, marker="o")
plt.title(f"{model_name} - Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)

loss_plot_path = perf_dir / f"{model_name.replace(' ', '_')}_loss.png"
plt.savefig(loss_plot_path)
plt.show()
plt.close()

# -----
# Evaluation on test set
# -----
model.eval()
predictions = []
ground_truths = []

start_inf_time = time.time()
with torch.no_grad():
    for videos, labels in test_loader:

```

```

        videos = videos.to(DEVICE)
        labels = labels.to(DEVICE)

        outputs = model(videos)
        batch_preds = torch.argmax(outputs, dim=1)

        predictions.extend(batch_preds.cpu().numpy())
        ground_truths.extend(labels.cpu().numpy())

inference_time = (time.time() - start_inf_time) / len(ground_truths)

# -----
# Compute evaluation metrics
# -----
acc = accuracy_score(ground_truths, predictions)
prec = precision_score(ground_truths, predictions, average="macro")
rec = recall_score(ground_truths, predictions, average="macro")
f1 = f1_score(ground_truths, predictions, average="macro")
cm = confusion_matrix(ground_truths, predictions)

# -----
# Classification report (per-class metrics)
# -----
class_report = classification_report(
    ground_truths,
    predictions,
    output_dict=True
)
class_report_df = pd.DataFrame(class_report).T

# ----- Print classification report -----
display(Markdown(f"### Classification Report - {model_name}"))
display(class_report_df.round(4))

# -----
# Compute model parameters and memory usage
# -----
num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

with tempfile.NamedTemporaryFile(delete=False) as tmp:
    torch.save(model.state_dict(), tmp.name)
    mem_mb = os.path.getsize(tmp.name) / (1024 ** 2)
os.remove(tmp.name)

# -----
# Metrics table (Pandas DataFrame) - PDF SAFE
# -----

```

```

metrics_df = pd.DataFrame([
    {
        "Model": model_name,
        "Accuracy": acc,
        "Precision (Macro)": prec,
        "Recall (Macro)": rec,
        "F1-score (Macro)": f1,
        "Training Time (s)": train_time,
        "Inference Time / Sample (s)": inference_time,
        "Model Parameters": num_params,
        "Memory Usage (MB)": mem_mb
    }
])

# ----- Print metrics -----
display(Markdown(f"### Model Performance Summary - {model_name}"))
display(metrics_df.round(4)) # PDF-safe replacement for .style

# -----
# Confusion matrix (SAVE + DISPLAY)
# -----
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title(f"{model_name} - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.tight_layout()

cm_path = cm_dir / f"{model_name.replace(' ', '_')}_cm.png"
plt.savefig(cm_path)
plt.show()
plt.close()

return metrics_df, class_report_df

```

```

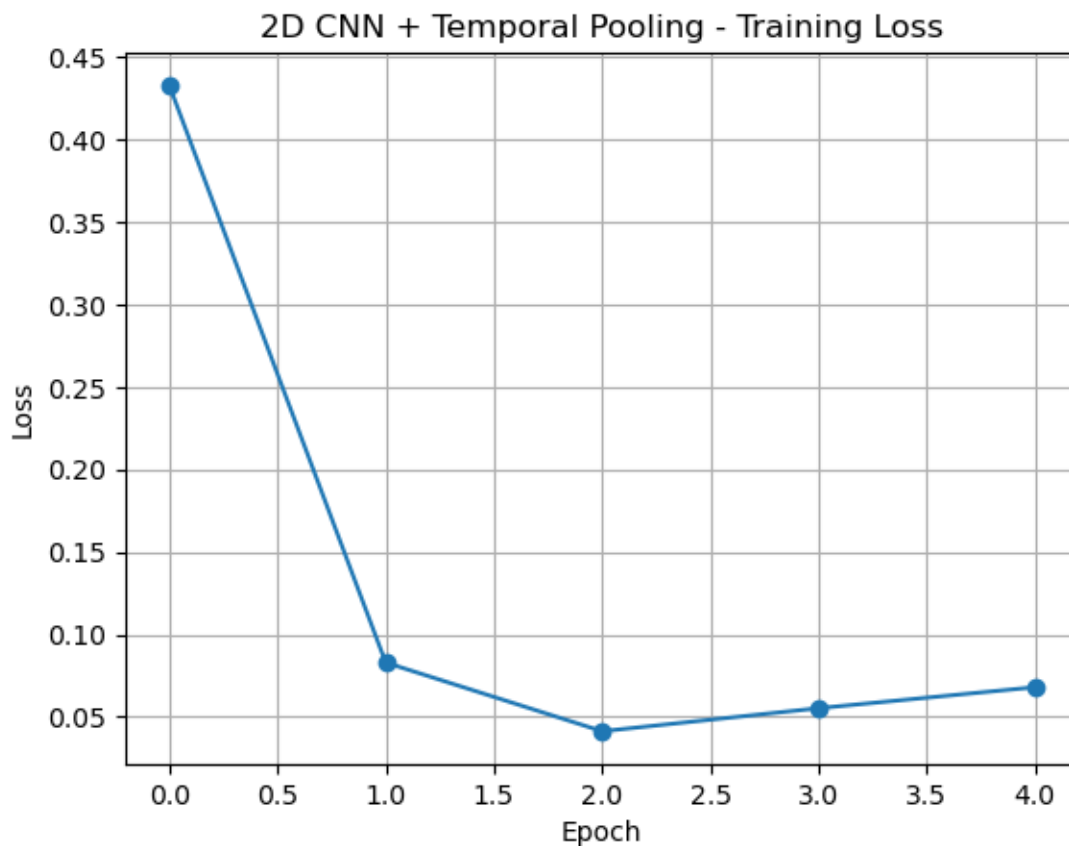
[22]: # =====
# RUN EXPERIMENTS - "2D CNN + Temporal Pooling"
# =====

metrics_2d, class_report_df_2d = train_and_evaluate(
    CNN2DTemporal(NUM_CLASSES),
    "2D CNN + Temporal Pooling"
)

```

Training model: 2D CNN + Temporal Pooling

Epoch [1/5] | Train Loss: 0.4331 | Val Loss: 0.0708
Epoch [2/5] | Train Loss: 0.0829 | Val Loss: 0.0348
Epoch [3/5] | Train Loss: 0.0412 | Val Loss: 0.0240
Epoch [4/5] | Train Loss: 0.0552 | Val Loss: 0.0230
Epoch [5/5] | Train Loss: 0.0679 | Val Loss: 0.0222

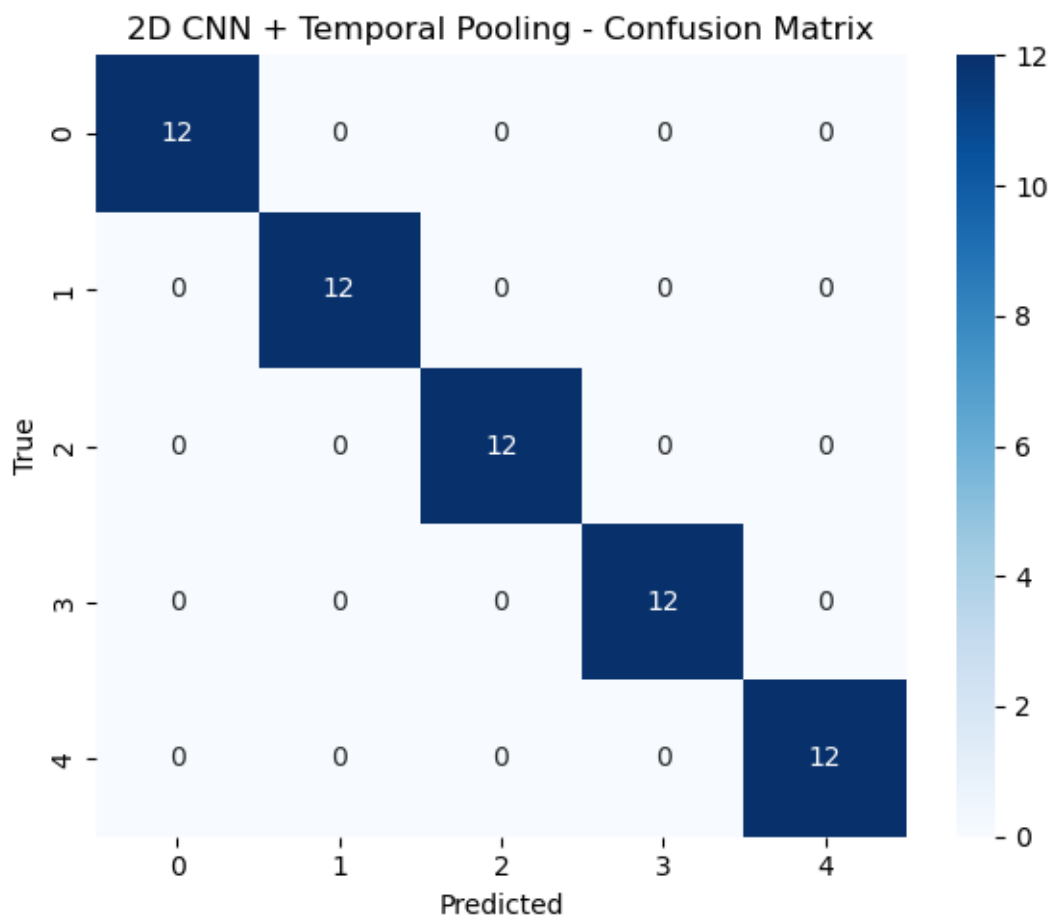


0.0.1 Classification Report — 2D CNN + Temporal Pooling

	precision	recall	f1-score	support
0	1.0	1.0	1.0	12.0
1	1.0	1.0	1.0	12.0
2	1.0	1.0	1.0	12.0
3	1.0	1.0	1.0	12.0
4	1.0	1.0	1.0	12.0
accuracy	1.0	1.0	1.0	1.0
macro avg	1.0	1.0	1.0	60.0
weighted avg	1.0	1.0	1.0	60.0

0.0.2 Model Performance Summary — 2D CNN + Temporal Pooling

	Model	Accuracy	Precision (Macro)	Recall (Macro)	\
0	2D CNN + Temporal Pooling	1.0	1.0	1.0	
	F1-score (Macro)	Training Time (s)	Inference Time / Sample (s)	\	
0	1.0	37.3042	0.0115		
	Model Parameters	Memory Usage (MB)			
0	11180101	42.7306			

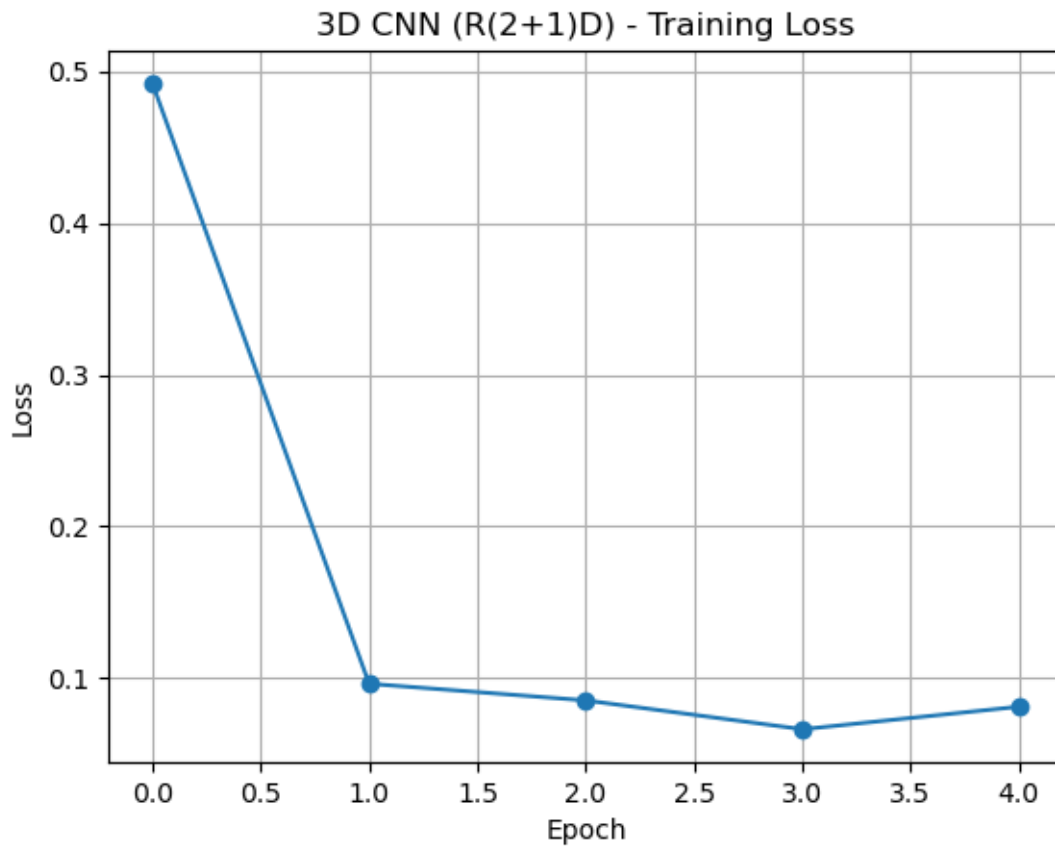


```
[23]: # =====
# RUN EXPERIMENTS - "3D CNN (R(2+1)D)"
# =====

metrics_3d, class_report_df_3d = train_and_evaluate(
    CNN3D(NUM_CLASSES),
    "3D CNN (R(2+1)D)"
)
```

Training model: 3D CNN (R(2+1)D)

Epoch [1/5] | Train Loss: 0.4920 | Val Loss: 0.0839
Epoch [2/5] | Train Loss: 0.0964 | Val Loss: 0.0302
Epoch [3/5] | Train Loss: 0.0855 | Val Loss: 0.0448
Epoch [4/5] | Train Loss: 0.0665 | Val Loss: 0.0255
Epoch [5/5] | Train Loss: 0.0813 | Val Loss: 0.0173



0.0.3 Classification Report — 3D CNN (R(2+1)D)

	precision	recall	f1-score	support
0	1.0	1.0	1.0	12.0
1	1.0	1.0	1.0	12.0
2	1.0	1.0	1.0	12.0
3	1.0	1.0	1.0	12.0
4	1.0	1.0	1.0	12.0
accuracy	1.0	1.0	1.0	1.0

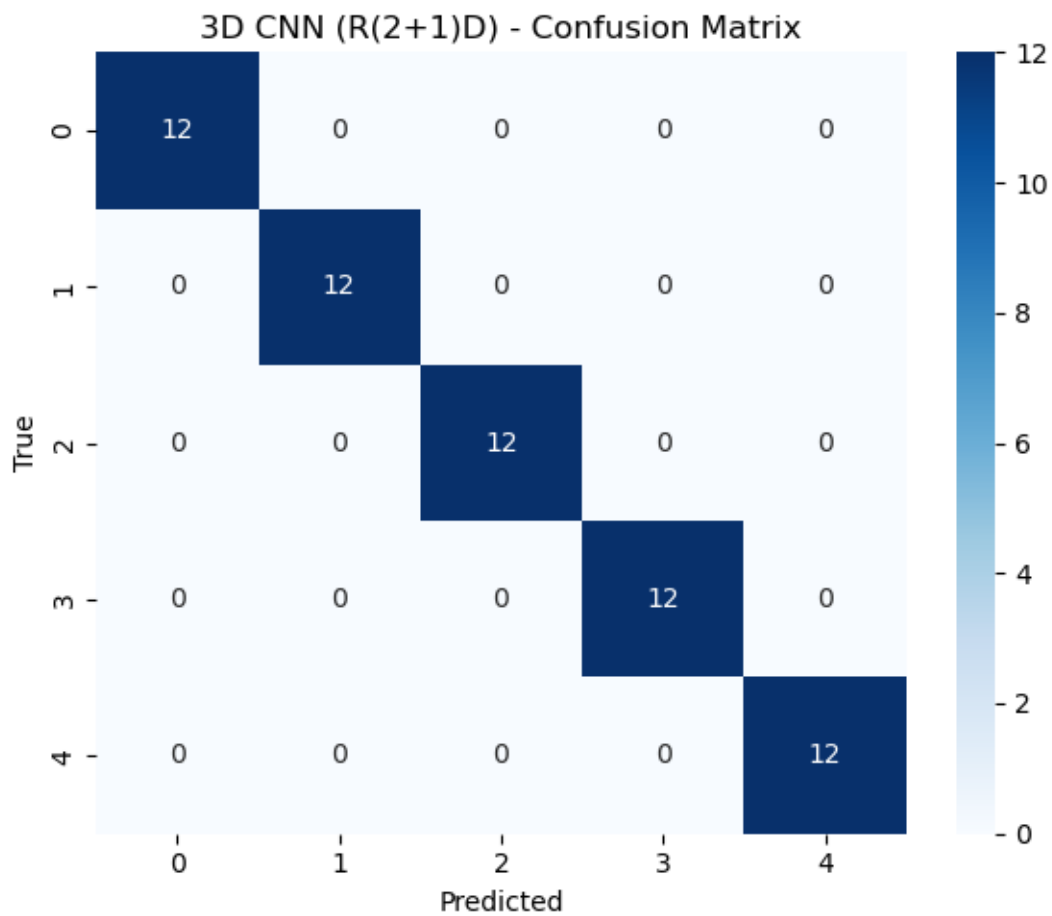
macro avg	1.0	1.0	1.0	60.0
weighted avg	1.0	1.0	1.0	60.0

0.0.4 Model Performance Summary — 3D CNN (R(2+1)D)

	Model	Accuracy	Precision (Macro)	Recall (Macro)	\
0	3D CNN (R(2+1)D)	1.0	1.0	1.0	

	F1-score (Macro)	Training Time (s)	Inference Time / Sample (s)	\
0	1.0	183.758	0.0249	

	Model Parameters	Memory Usage (MB)
0	31303714	119.5918



```
[24]: # =====
# DETAILED MODEL COMPARISON REPORT (PDF-SAFE)
# =====

def compare_models(metrics_2d_df, metrics_3d_df, save_path=None):
```

```

"""
Generate a detailed comparison report between 2D CNN and 3D CNN models
and optionally save metrics to a spreadsheet.

Includes:
- Bar charts (Accuracy, Training Time, Inference Time, Memory, Model Size)
- Colored textual summary (PDF-safe using Markdown)
- Tabulated metrics with ranks
- Optional export to CSV / Excel

Args:
    metrics_2d_df (pd.DataFrame): Metrics DataFrame for 2D CNN (single row)
    metrics_3d_df (pd.DataFrame): Metrics DataFrame for 3D CNN (single row)
    save_path (str, optional): Path to save spreadsheet (.csv or .xlsx)
"""

from IPython.display import Markdown, display

# -----
# Step 1: Extract metrics
# -----
metrics_2d = metrics_2d_df.iloc[0]
metrics_3d = metrics_3d_df.iloc[0]

models = ["2D CNN", "3D CNN"]

accuracy = [metrics_2d["Accuracy"], metrics_3d["Accuracy"]]
train_time = [metrics_2d["Training Time (s)"], metrics_3d["Training Time_
↪(s)"]]
inf_time = [
    metrics_2d["Inference Time / Sample (s)"],
    metrics_3d["Inference Time / Sample (s)"]
]
mem_usage = [metrics_2d["Memory Usage (MB)"], metrics_3d["Memory Usage_
↪(MB)"]]
model_params = [metrics_2d["Model Parameters"], metrics_3d["Model_
↪Parameters"]]

# -----
# Step 2: Plot bar charts
# -----
def plot_bar(values, title, ylabel):
    plt.figure(figsize=(6, 4))
    bars = plt.bar(models, values)
    plt.title(title)
    plt.ylabel(ylabel)
    plt.grid(axis="y", linestyle="--", alpha=0.7)

```

```

        for bar, value in zip(bars, values):
            plt.text(
                bar.get_x() + bar.get_width() / 2,
                bar.get_height(),
                f"{value:.4f}",
                ha="center",
                va="bottom",
                fontsize=10
            )
    plt.show()

plot_bar(accuracy, "Model Accuracy Comparison", "Accuracy")
plot_bar(train_time, "Training Time Comparison", "Time (s)")
plot_bar(inf_time, "Inference Time per Sample Comparison", "Time (s)")
plot_bar(model_params, "Model Size (Number of Parameters)", "Parameters")
plot_bar(mem_usage, "Memory Footprint Comparison", "Memory (MB)")

# -----
# Step 3: Colored textual summary (PDF-safe Markdown)
# -----
def color_text(name, value, values_list, reverse=False):
    is_best = value == (min(values_list) if reverse else max(values_list))
    color = "green" if is_best else "red"
    return f"**{name}: {value:.4f}** ({'Best' if is_best else ''})"

display(Markdown("### Accuracy Comparison"))
display(Markdown(
    f"{color_text('2D CNN Accuracy', accuracy[0], accuracy)} \n"
    f"{color_text('3D CNN Accuracy', accuracy[1], accuracy)}"
))

display(Markdown("### Computational Efficiency"))
display(Markdown(
    f"{color_text('2D CNN Training Time (s)', train_time[0], train_time, ↵
↵reverse=True)} \n"
    f"{color_text('3D CNN Training Time (s)', train_time[1], train_time, ↵
↵reverse=True)} \n"
    f"{color_text('2D CNN Inference Time / Sample (s)', inf_time[0], ↵
↵inf_time, reverse=True)} \n"
    f"{color_text('3D CNN Inference Time / Sample (s)', inf_time[1], ↵
↵inf_time, reverse=True)}"
))

display(Markdown("### Model Complexity"))
display(Markdown(
    f"{color_text('2D CNN Parameters', model_params[0], model_params)} \n"

```

```

        f"{color_text('3D CNN Parameters', model_params[1], model_params)} \n"
        f"{color_text('2D CNN Memory (MB)', mem_usage[0], mem_usage,
↪reverse=True)} \n"
        f"{color_text('3D CNN Memory (MB)', mem_usage[1], mem_usage,
↪reverse=True)}"
    ))

    # -----
    # Step 4: Combine metrics and compute ranks
    # -----
    combined_df = pd.concat([metrics_2d_df, metrics_3d_df], ignore_index=True)

    higher_better = [
        "Accuracy",
        "Precision (Macro)",
        "Recall (Macro)",
        "F1-score (Macro)",
        "Model Parameters" # larger params considered "better" may be
↪subjective
    ]

    lower_better = [
        "Training Time (s)",
        "Inference Time / Sample (s)",
        "Memory Usage (MB)"
    ]

    for col in higher_better:
        combined_df[f"{col} Rank"] = combined_df[col].rank(ascending=False,
↪method="min").astype(int)

    for col in lower_better:
        combined_df[f"{col} Rank"] = combined_df[col].rank(ascending=True,
↪method="min").astype(int)

    # -----
    # Step 5: Display table PDF-safe
    # -----
    display(Markdown("### Metrics Comparison with Rankings"))
    display(combined_df.round(4))

    # -----
    # Step 6: Save to spreadsheet (CSV / Excel)
    # -----
    if save_path is not None:
        os.makedirs(os.path.dirname(save_path), exist_ok=True)

```

```

    if save_path.endswith(".csv"):
        combined_df.to_csv(save_path, index=False)
    elif save_path.endswith(".xlsx"):
        combined_df.to_excel(save_path, index=False)
    else:
        raise ValueError("save_path must end with .csv or .xlsx")

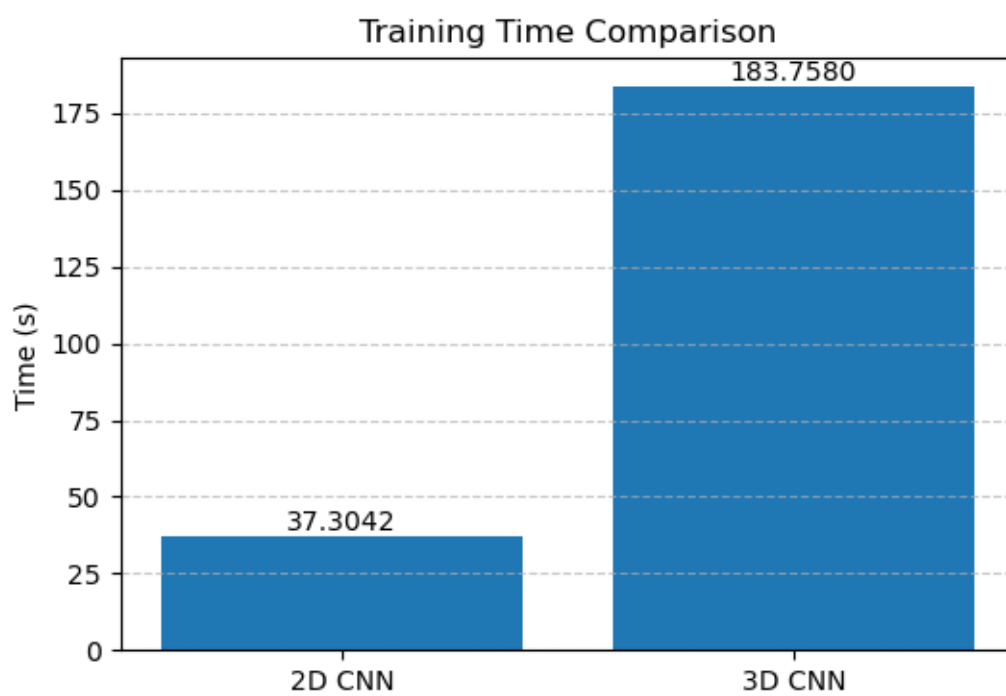
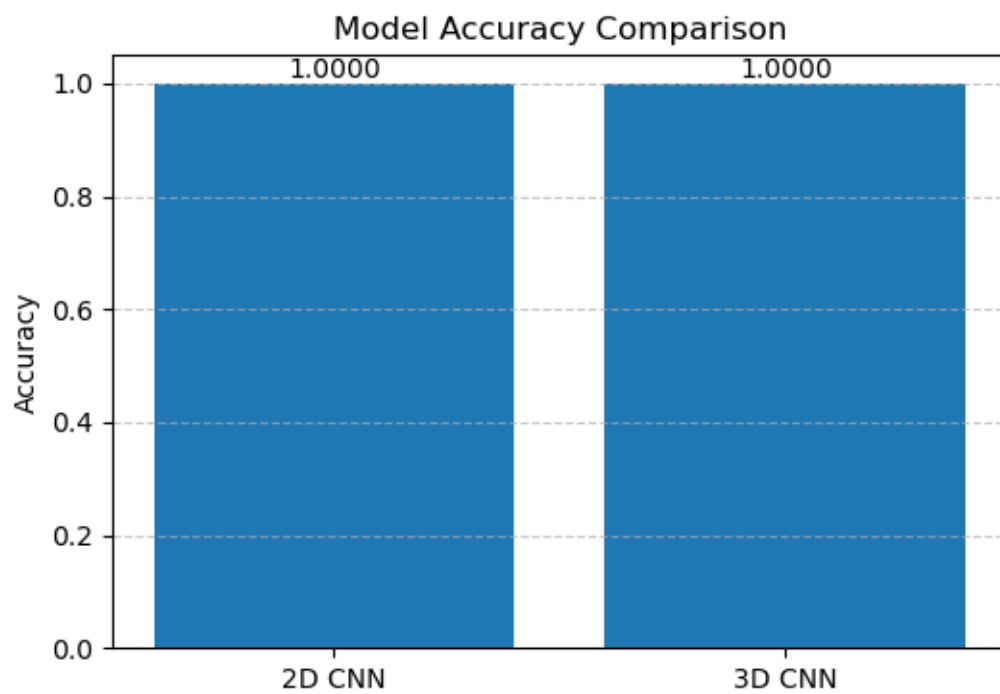
    print(f"\n Metrics successfully saved to: {save_path}")

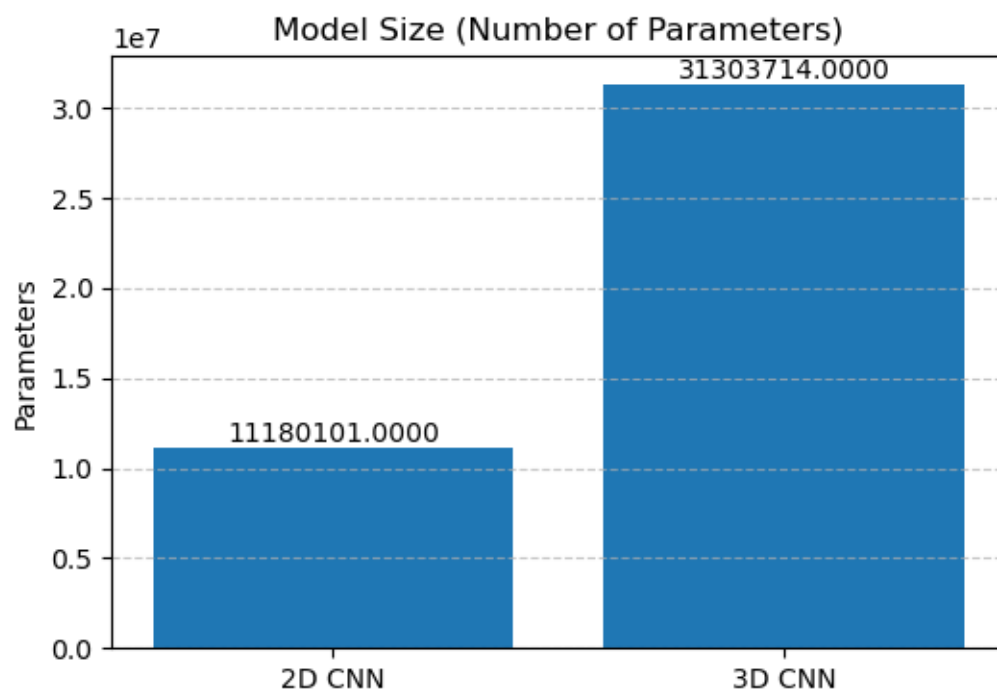
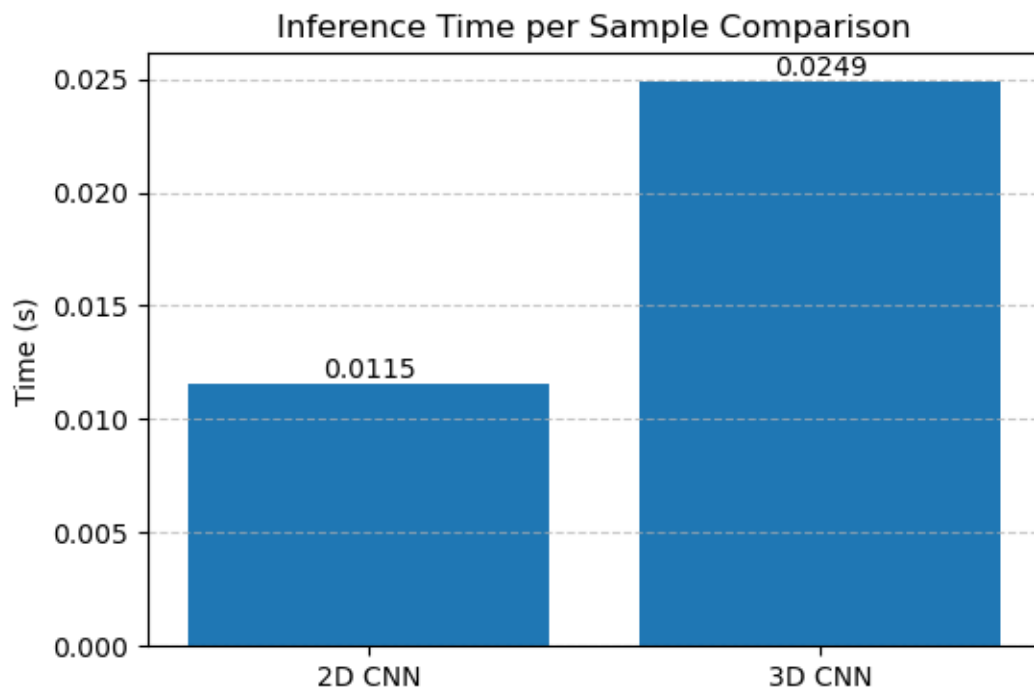
# -----
# Step 7: Final conclusion
# -----
display(Markdown("### Final Conclusion"))
display(Markdown(
    "2D CNN provides efficient inference and lower training cost, "
    "while 3D CNN captures temporal dynamics at higher computational_
expense."
))

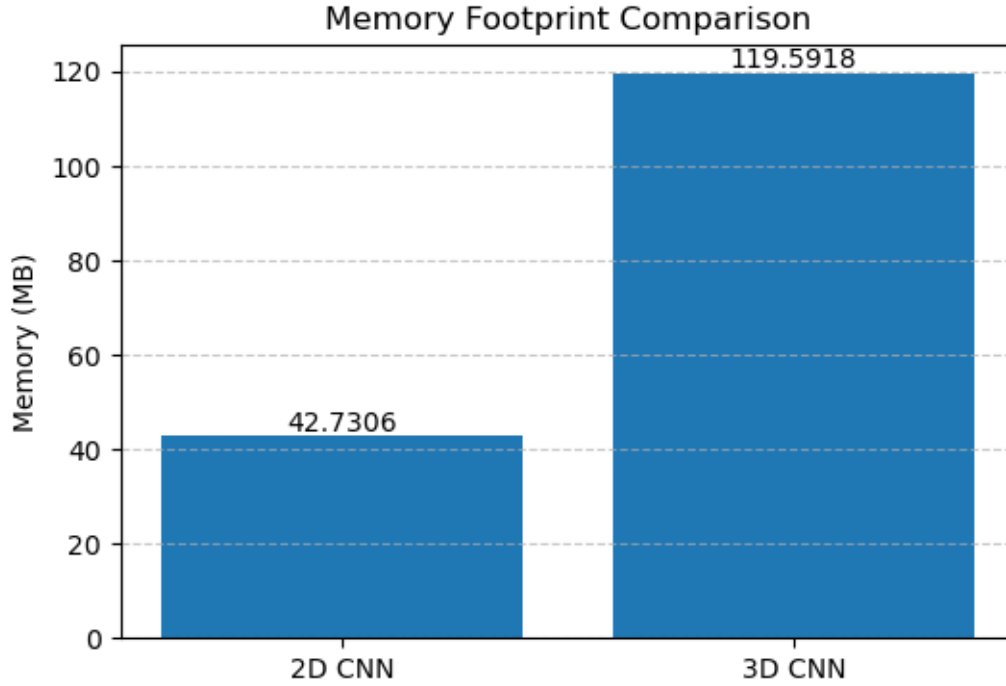
print("\n Experiment completed successfully")

# =====
# USAGE
# =====
compare_models(
    metrics_2d,
    metrics_3d,
    save_path=f"{PROJECT_ROOT}/results/deep_learning_comparison_metrics.csv"
)

```







0.0.5 Accuracy Comparison

2D CNN Accuracy: 1.0000 (Best)

3D CNN Accuracy: 1.0000 (Best)

0.0.6 Computational Efficiency

2D CNN Training Time (s): 37.3042 (Best)

3D CNN Training Time (s): 183.7580 ()

2D CNN Inference Time / Sample (s): 0.0115 (Best)

3D CNN Inference Time / Sample (s): 0.0249 ()

0.0.7 Model Complexity

2D CNN Parameters: 11180101.0000 ()

3D CNN Parameters: 31303714.0000 (Best)

2D CNN Memory (MB): 42.7306 (Best)

3D CNN Memory (MB): 119.5918 ()

0.0.8 Metrics Comparison with Rankings

	Model	Accuracy	Precision (Macro)	Recall (Macro)	\
0	2D CNN + Temporal Pooling	1.0	1.0	1.0	
1	3D CNN (R(2+1)D)	1.0	1.0	1.0	

F1-score (Macro)	Training Time (s)	Inference Time / Sample (s)	\
------------------	-------------------	-----------------------------	---

0	1.0	37.3042	0.0115
1	1.0	183.7580	0.0249

	Model Parameters	Memory Usage (MB)	Accuracy Rank	Precision (Macro) Rank	\
0	11180101	42.7306	1	1	
1	31303714	119.5918	1	1	

	Recall (Macro) Rank	F1-score (Macro) Rank	Model Parameters Rank	\
0	1	1	2	
1	1	1	1	

	Training Time (s) Rank	Inference Time / Sample (s) Rank	\
0	1	1	
1	2	2	

	Memory Usage (MB) Rank
0	1
1	2

Metrics successfully saved to: /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/deep_learning_comparison_metrics.csv

0.0.9 Final Conclusion

2D CNN provides efficient inference and lower training cost, while 3D CNN captures temporal dynamics at higher computational expense.

Experiment completed successfully

[]: